DoDo Bot — Perception Stack

Obstacle Detection Model — COCO + YOLOv8

This report documents the full process of training a baseline obstacle-detection model for DoDo Bot, including every error encountered and how it was resolved, so the pipeline can be understood and reproduced end-to-end.

Metric	Value
Model	YOLOv8-nano (fine-tuned from pretrained COCO weights)
Training images	2,000
Epochs	20
Hardware	Google Colab, Tesla T4 GPU
Training time	~18 minutes
Overall mAP50	0.482
person mAP50	0.766
dining table mAP50	0.563
chair mAP50	0.547

Objective

Train a lightweight object detector that recognizes restaurant-relevant obstacle classes (person, chair, dining table, backpack, handbag) without needing physical robot hardware yet, using a public dataset (COCO) as a bootstrap — the same approach outlined in the original Day 1 plan.

Step-by-Step Walkthrough

1. Attempted dataset download via fiftyone

The original plan used the `fiftyone` library to auto-download a filtered COCO subset (only the 5 target classes). This failed with:

```
ImportError: cannot import name '_Ink' from 'PIL._typing'
```

This is a bug inside fiftyone 1.19.0's own bundled image-handling code — it references a Pillow internal attribute that doesn't exist in any public Pillow release. Upgrading or downgrading Pillow did not fix it, since the bug is in fiftyone's code, not Pillow's.

2. Switched to local filtering with pycocotools

Since the full COCO dataset was already downloaded locally, fiftyone was dropped entirely. Instead, `pycocotools` was used locally (on the laptop, not Colab) to filter the full dataset down to only images containing the 5 target classes, then export a smaller `coco_subset_yolo` folder in YOLO format.

3. Uploaded the filtered subset to Colab

The resulting folder was zipped (~315MB) and uploaded directly into Colab's session storage.

4. Hit a zip-read error from unzipping too early

Running the unzip command immediately after starting the upload produced:

```
unzip: cannot find zipfile directory in one of coco_subset_yolo.zip...
```

The upload had not finished when the unzip command ran. Waiting for Colab's upload progress indicator to fully disappear before unzipping resolved this — not a code issue, a timing issue.

5. Verified folder structure

After a successful unzip, the dataset folder contained `images/`, `labels/`, and `dataset.yaml`, with images and labels nested inside `val/` subfolders (2,000 images, 2,000 matching label files).

6. Fixed a missing train: path in dataset.yaml

Inspecting `dataset.yaml` showed it only had a `val:` key and no `train:` key, which YOLOv8 requires. It was rewritten to point both `train:` and `val:` at the same `images/val` folder — sufficient for a working baseline tonight, though a proper held-out validation split is recommended before treating the accuracy numbers as a true generalization measure.

7. Trained YOLOv8n

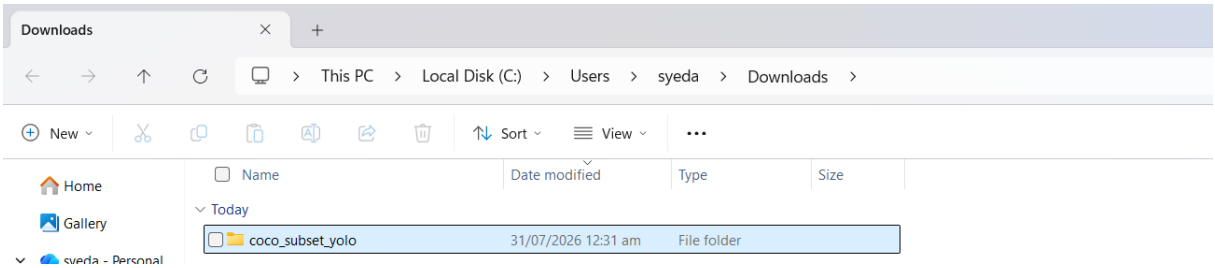
Training ran for 20 epochs on a Tesla T4 GPU in Colab, completing in about 18 minutes with no errors. Loss values decreased steadily and mAP50 rose from 0.012 after epoch 1 to 0.482 by epoch 20.

8. Evaluated results and exported weights

The trained weights (best.pt, ~6.5MB) were downloaded locally, along with a results plot and a full per-class breakdown (see Appendix).

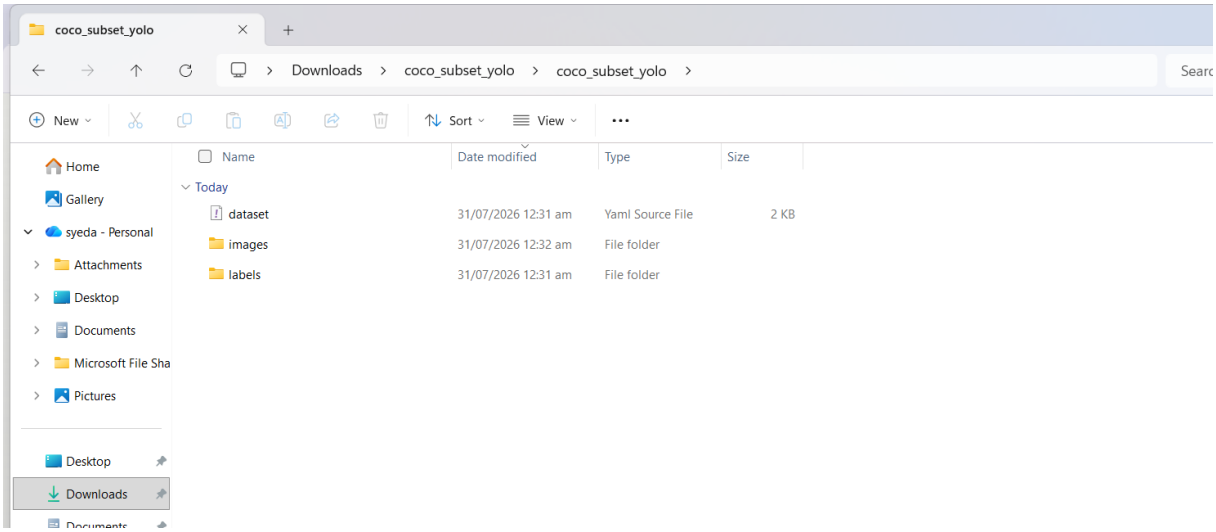
Process Screenshots

Screenshot 1 — Confirming the COCO download location



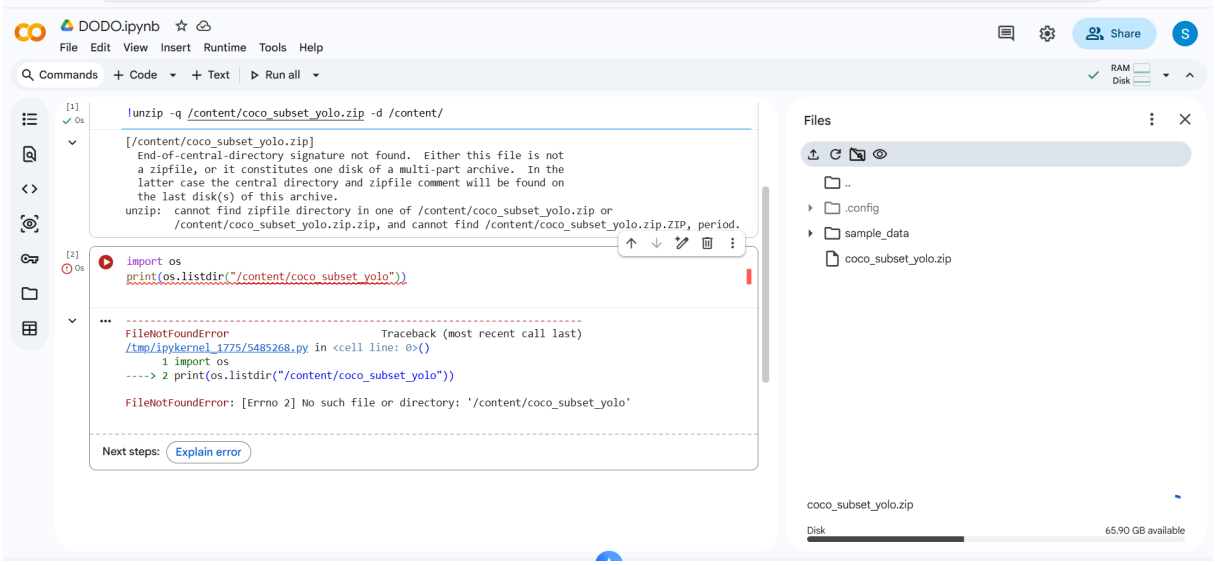
File Explorer showing the coco_subset_yolo folder in Downloads, before upload to Colab.

Screenshot 2 — Verifying the filtered dataset structure



Contents of coco_subset_yolo: dataset.yaml, images/, and labels/ — confirmed complete before zipping.

Screenshot 3 — The zip-read error in Colab



Colab notebook showing the 'cannot find zipfile directory' error, caused by unzipping before the upload finished.

Appendix — Full Training Results

Final validation results after 20 epochs (best.pt), restaurant-relevant classes highlighted. Full per-class table available in Day1_ObstacleDetection/results/training_log.txt in the repo.

Class	Instances	Precision	Recall	mAP50	mAP50-95
person	7446	0.872	0.632	0.766	0.540
dining table	490	0.764	0.467	0.563	0.422
chair	1172	0.673	0.474	0.547	0.373
backpack	327	0.586	0.220	0.272	0.163
handbag	458	0.564	0.114	0.196	0.118
ALL (79 classes)	**19424**	**0.676**	**0.440**	**0.482**	**0.346**

Known Limitation

The final dataset.yaml ended up listing 79 COCO classes rather than the 5 originally targeted, because the fiftyone class-filtering step never actually executed (it was replaced by the pycocotools approach, but the exported zip in use turned out to be a broader export). The model still performs reasonably on the 5 restaurant-relevant classes, but retraining on a dataset limited strictly to those 5 classes would likely improve accuracy further, since the model wouldn't spend capacity learning to distinguish 74 irrelevant classes.

Next Steps

Day 2: run this trained model against a restaurant video (frame-by-frame) to check real-world obstacle detection and floor-level object detection. Day 3: build the gesture recognition classifier. Separately: implement the ROS2 distance/safety-zone node (0.3m threshold).